

Lezione 8: Scheduling CPU

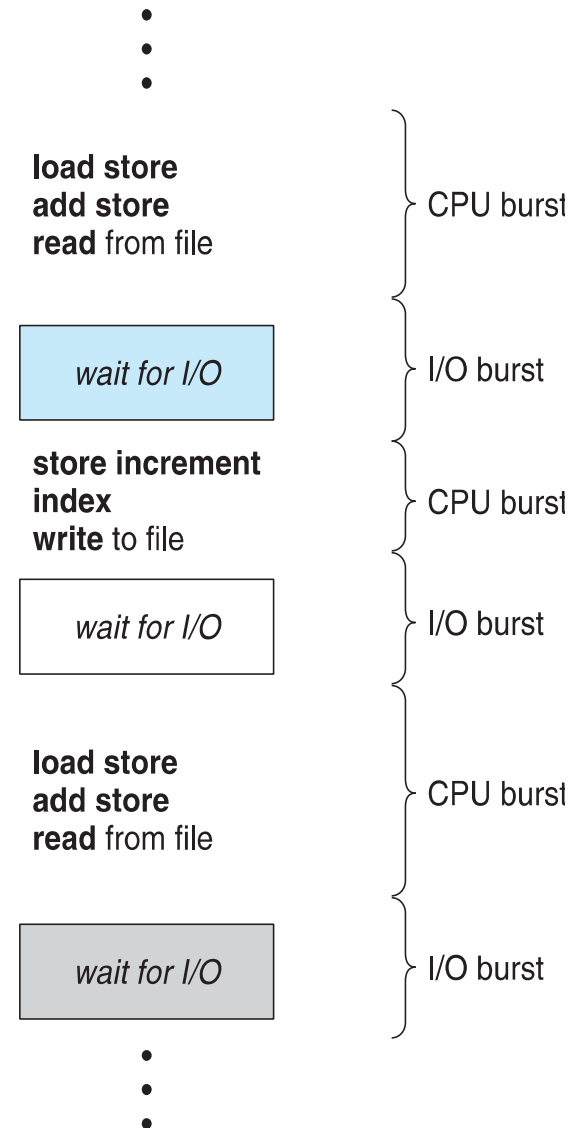


Obiettivi

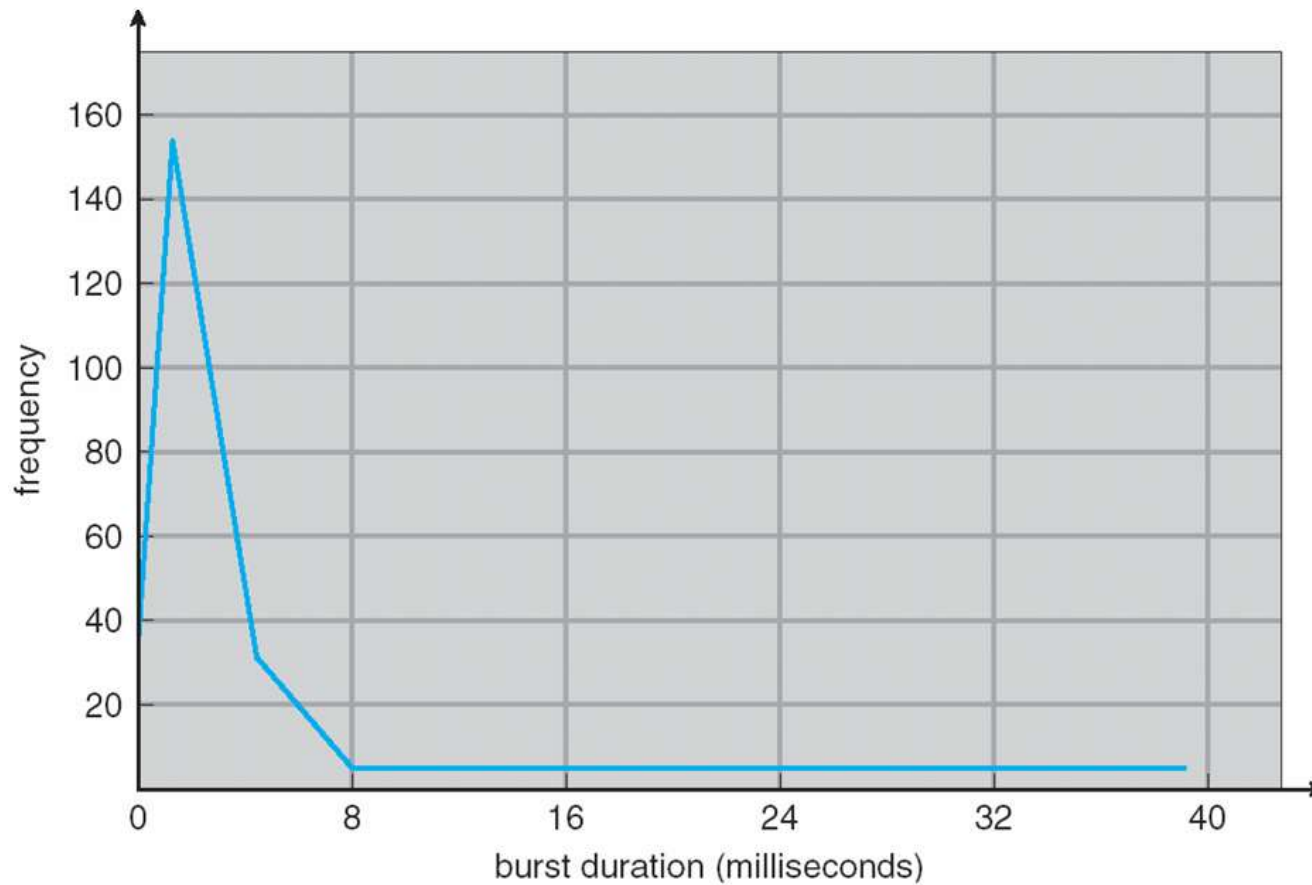
- Introdurre il concetto dello scheduling CPU
- Descrivere algoritmi di scheduling di CPU
- Discutere criteri di valutazione degli algoritmi di scheduling
- Esaminare algoritmi di scheduling algorithms di vari SO

Concetti di Base

- Massimo utilizzo di CPU con multiprogramming
- Cicli CPU–I/O Burst –
L'esecuzione dei processi
consiste di **cicli** di esecuzione
CPU e attese I/O
- **CPU burst** seguiti da **I/O burst**
- Distribuzione di CPU burst è una
problematica importante



Istogramma dei tempi di CPU-burst



CPU Scheduler

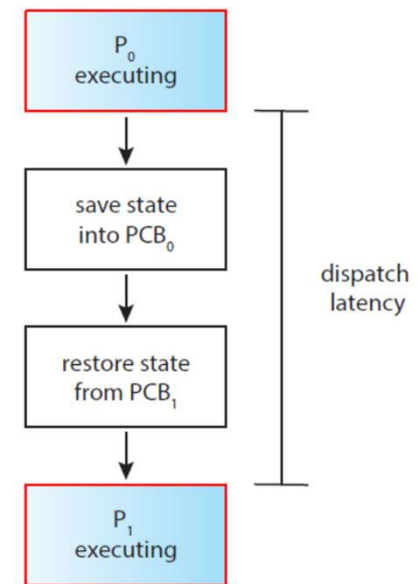
- **Short-term scheduler** seleziona tra processi nella coda ready e alloca la CPU a uno di loro
 - Le code possono essere ordinate in modi diversi
- Le decisioni di CPU scheduling occorrono quando i processi:
 1. Passano dallo stato running a waiting (es. I/O req o wait)
 2. Passano dallo stato running a ready (es. interrupt)
 3. Passano da waiting a ready (es. I/O completo)
 4. Terminano (es. terminazione)
- In situazioni 1 e 4 processo cede il controllo (non prelazione)
- Schemi senza prelazione (**nonpreemptive**) o con prelazione (**preemptive**)
 - Scheduling senza prelazione (processo in esecuzione fino a fine o wait)
 - Scheduling con prelazione
 - Problema accesso a dati condivisi
 - Problema prelazione in modalità kernel
 - Problema interruzioni durante attività curciali del SO

Dispatcher

- ❑ Dispatcher dà il controllo della CPU ai processi selezionati dallo short-term scheduler e richiede:
 - ❑ switching del contesto
 - ❑ switching allo user mode
 - ❑ salto alla corretta locazione del programma utente per ripartire con quel programma
- ❑ **Latenza di dispatch** – tempo necessario al dispatcher per fermare un processo e mandarne un altro in running
 - ❑ Più breve possibile
- ❑ Context switch:

```
vmstat 1 3  
  
-----cpu-----  
24  
225  
339
```

in: The number of interrupts per second, including the clock.
cs: The number of context switches per second.



Criteri di Scheduling

- ❑ **Utilizzo CPU** –percentuale di CPU utilizzata; occorre mantenere la CPU occupata (top command).
- ❑ **Throughput** – numero di processi che completano l'esecuzione per unità di tempo
- ❑ **Turnaround time** – tempo di completamento di processo
 - ❑ tempo totale per eseguire un processo
 - ❑ accesso memoria + coda ready + CPU + I/O
- ❑ **Waiting time** – tempo di attesa per un processo nella coda ready
- ❑ **Response time** – tempo che intercorre tra una richiesta e la prima risposta
 - ❑ Importante per sistemi interattivi, quando l'elaborazione continua dopo un output
 - ❑ Alternativa al tempo di turnaround che prevede l'esecuzione completa

Scheduling: Criteri di Ottimizzazione

- ❑ Massimo utilizzo CPU
 - ❑ Massimo throughput
 - ❑ Minimo tempo di turnaround
 - ❑ Minimo tempo di waiting
 - ❑ Minimo tempo di risposta
-
- ❑ In molti sistemi (desktop, laptop) è più importante minimizzare la varianza dei tempi di risposta che il tempo medio

First- Come, First-Served (FCFS) Scheduling

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

- Assumendo che i processi nell'ordine: P_1 , P_2 , P_3
Il Gantt Chart dello schedule è:



- Tempi di attesa per $P_1 = 0$; $P_2 = 24$; $P_3 = 27$
- Media del tempo di attesa: $(0 + 24 + 27)/3 = 17$

FCFS Scheduling

Assumendo i processi nell'ordine:

$$P_2, P_3, P_1$$

□ Il Gantt diventa:



- Tempi di attesa $P_1 = 6; P_2 = 0; P_3 = 3$
- Tempo di attesa medio: $(6 + 0 + 3)/3 = 3$
- Molto meglio di prima
- **Effetto Convoy** - processi brevi dietro i processi lunghi
 - Considera un processo CPU-bound e tanti I/O-bound
- Sbrigare i processi brevi per migliorare i tempi di risposta

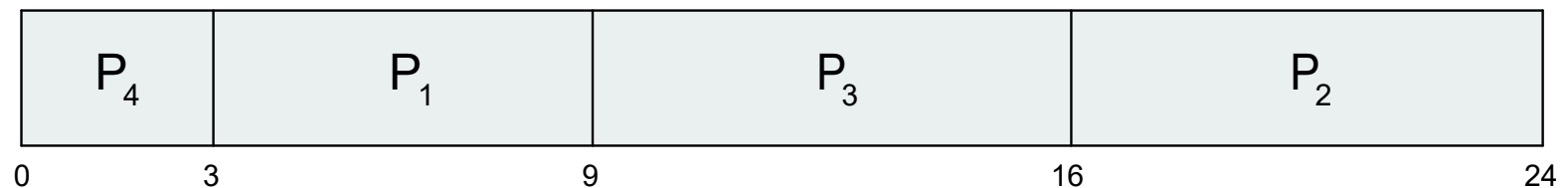
Shortest-Job-First (SJF) Scheduling

- ❑ Associa ad ogni processo la lunghezza del prossimo CPU burst
 - ❑ Lunghezza usata per schedulare il processo con burst più breve
- ❑ SJF è ottimale – minima attesa media per un insieme di processi
 - ❑ Difficile conoscere la lunghezza della prossima richiesta di CPU
 - ❑ Soluzione: uso di stime basate sul comportamento passato ed esecuzione del processo con il minor tempo di esecuzione stimato
- ❑ Algoritmo particolarmente indicato per l'esecuzione batch dei job per i quali i tempi di esecuzione sono conosciuti a priori
- ❑ Lo scheduler dovrebbe utilizzare questo algoritmo quando nella coda di input risiedono job di uguale importanza

Esempio di SJF

<u>Process</u>	<u>Burst Time</u>
P_1	6
P_2	8
P_3	7
P_4	3

□ SJF scheduling chart

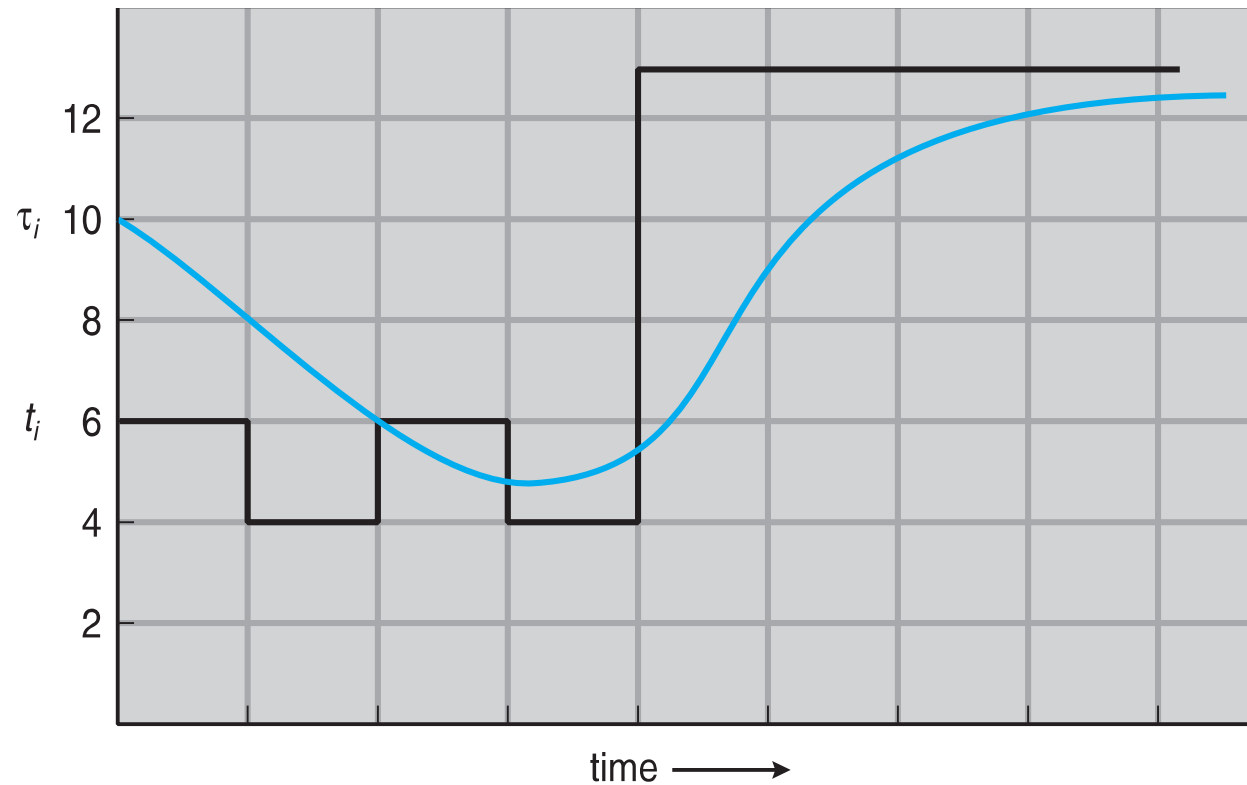


□ Tempo di attesa medio = $(3 + 16 + 9 + 0) / 4 = 7$

Lunghezza del prossimo CPU Burst

- Si può solo stimare – simile alla precedente
 - Prendi il processo con il CPU burst stimato come il più breve
- Utilizzare la lunghezza del CPU burst precedente, usando la **media esponenziale** delle lunghezze precedenti
 1. t_n = actual length of n^{th} CPU burst
 2. τ_{n+1} = predicted value for the next CPU burst
 3. $\alpha, 0 \leq \alpha \leq 1$
 4. Define: $\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$.
- Solitamente α settato a $\frac{1}{2}$
- Versione preemptive chiamata **shortest-remaining-time-first**

Lunghezza del prossimo CPU Burst



CPU burst (t_i)	6	4	6	4	13	13	13	...
"guess" (τ_i)	10	8	6	6	9	11	12	...

Esempi di Exponential Averaging

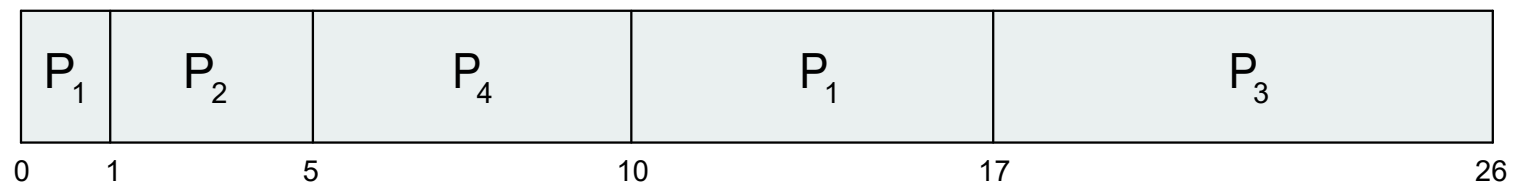
- $\alpha = 0$
 - $\tau_{n+1} = \tau_n$
 - Storia recente non conta
- $\alpha = 1$
 - $\tau_{n+1} = \alpha t_n$
 - Conta solo l'ultimo CPU burst
- Espandendo la formula si ottiene:
$$\begin{aligned}\tau_{n+1} = & \alpha t_n + (1 - \alpha)\alpha t_{n-1} + \dots \\ & + (1 - \alpha)^j \alpha t_{n-j} + \dots \\ & + (1 - \alpha)^{n+1} \tau_0\end{aligned}$$
- Sia α che $(1 - \alpha)$ sono minori o uguali ad 1, ogni termine successiva ha minor peso del predecessore

Esempio di Shortest-remaining-time-first

- Si considerano diversi tempi di arrivo e si introduce la prelazione

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

- *Preemptive* SJF Gantt Chart



- Tempo attesa medio = $[(10-1)+(1-1)+(17-2)+5-3]/4 = 26/4 = 6.5$ msec
- Senza prelazione 8.75 msec

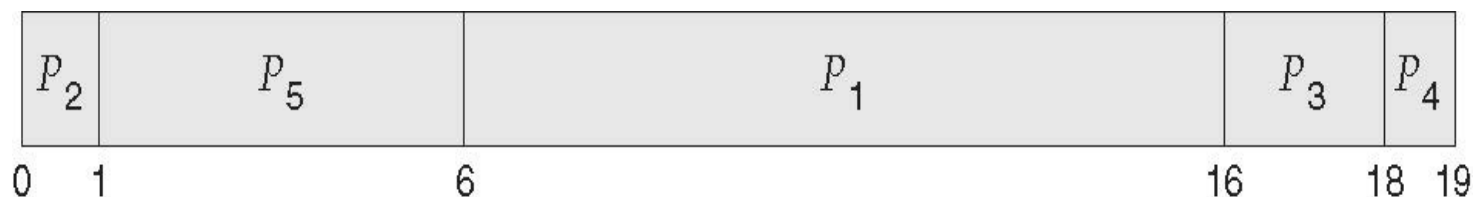
Scheduling con Priorità

- Numero di priorità (intero) associato ad ogni processo
- CPU allocata al processo con più alta priorità (minor intero $\equiv$ maggiore priorità)
 - Preemptive
 - Nonpreemptive
- SJF è un priority scheduling con priorità inversamente proporzionale alla predizione del prossimo CPU
- Problema $\equiv$ **Starvation** – processi a bassa priorità mai eseguiti
- Soluzione $\equiv$ **Aging** – al passare del tempo aumenta la priorità del processo

Esempio di Priority Scheduling

<u>Process</u>	<u>Burst Time</u>	<u>Priority</u>
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

□ Priority scheduling Gantt Chart



□ Tempo di attesa medio = 8.2 msec

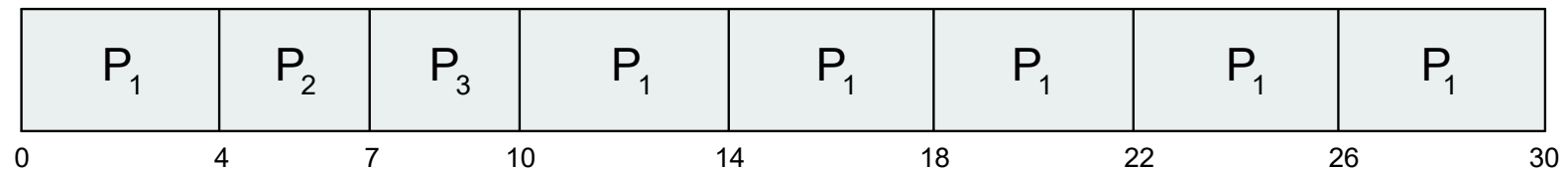
Round Robin (RR)

- Ogni processo riceve una piccola unità di CPU (**time quantum** q), di solito 10-100 msec. A tempo scaduto il processo viene prelazionato e messo alla fine della coda ready
- Con n processi in coda ready e time quantum q , ogni processo riceve $1/n$ di CPU time in chunk di q unità di tempo
- Tempi di attesa inferiori a $(n-1)q$ unità di tempo
 - Es. 5 processi con $q = 20$ millisec prende 20 millisec ogni 100
- Un timer interrompe ad ogni quanto per schedulare il processo successivo
- Prestazioni
 - q largo $\Rightarrow$ FIFO
 - q piccolo $\Rightarrow q$ grande rispetto al context switch altrimenti l'overhead è troppo alto

Esempio di RR con Quanto = 4

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

□ Gantt chart:



- Tipicamente maggiore turnaround medio di SJF, ma migliore **risposta**
- q deve essere grande rispetto al tempo di context switch
- q di solito tra 10ms e 100ms, context switch < 10 microsec

Quanto di Tempo e Tempo di Context Switch

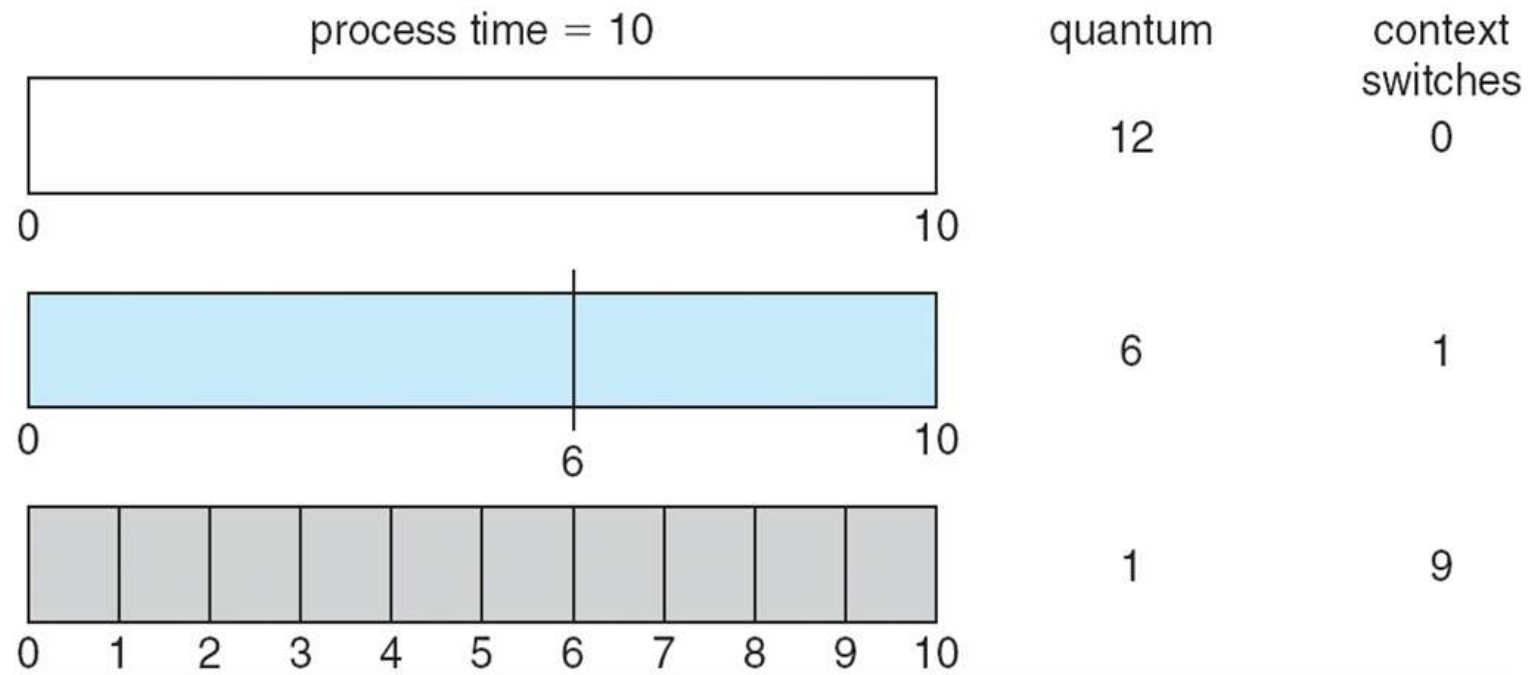
Assumendo un processo di 10 unità di tempo

Se il quanto è 12 il processo completa senza context switch

Se il quanto è 6 ne occorre uno

Se il quanto è 1 ne occorrono 9

Se cs è 10% del quanto, per ogni quanto si perde 10% di CPU

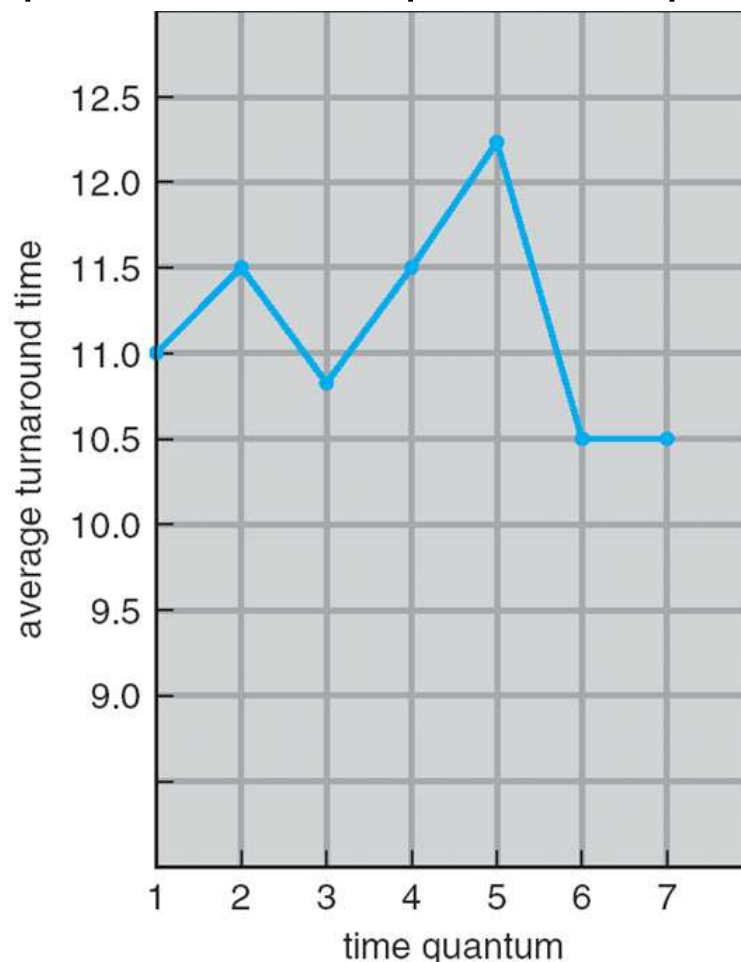


Tempo di Turnaround varia con il Time Quantum

Tempo di turnaround varia con la dimensione del quanto e non incrementa sempre con la dimensione

- es. 10 unità tempo se 1 q allora 29 turn se 10 q allora 20 turn

Dipende dal tempo di completamento dei processi



process	time
P_1	6
P_2	3
P_3	1
P_4	7

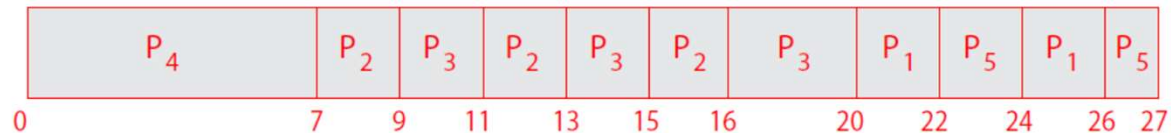
Regola empirica: l'80% dei CPU burst dovrebbero essere più corti di q

RR con Priorità

Si segue la priorità, processi con pari priorità con RR

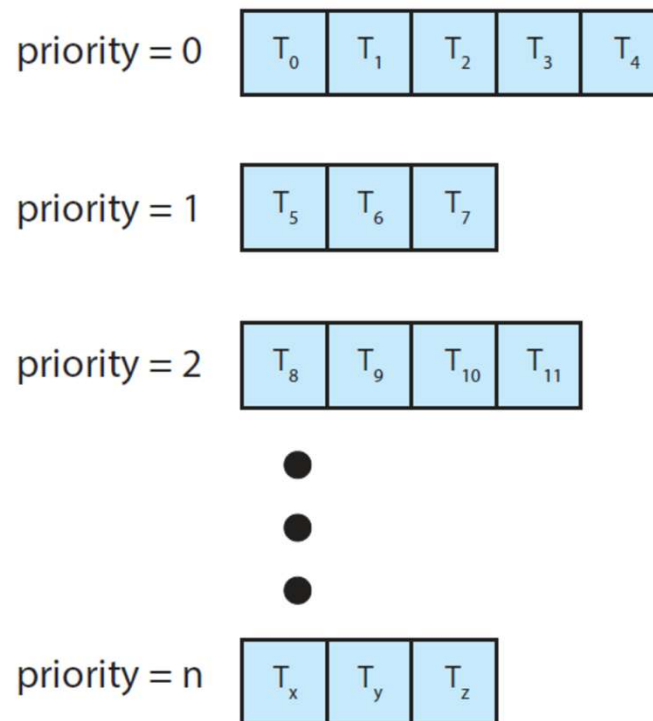
<u>Process</u>	<u>Burst Time</u>	<u>Priority</u>	millisecondi
P_1	4	3	
P_2	5	2	
P_3	8	2	
P_4	7	1	
P_5	3	3	

Quantum 2 millisecondi



Code Multiple

- Se una sola coda $O(n)$ per cercare le priorità
- Comodo avere code separate per le differenti priorità
- Assegnazione delle code ai processi
 - di solito statico e processo rimanente in quella coda
- Ogni coda ha il suo scheduling

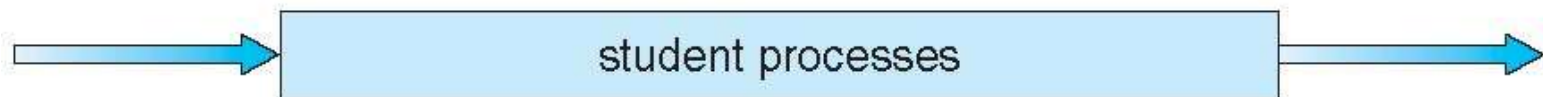
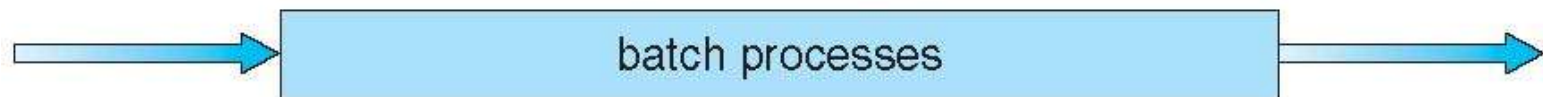
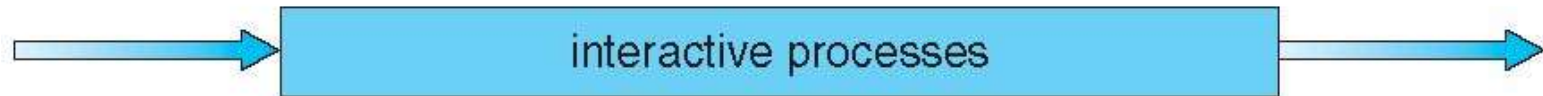
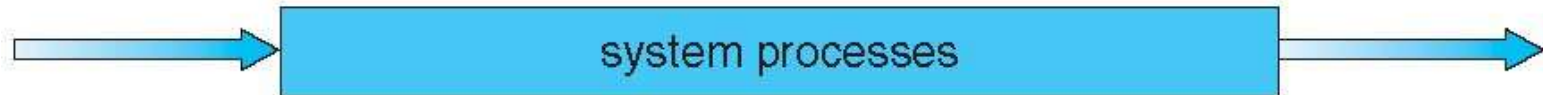


Code Multiple

- Si può usare per partizionare processi in base al tipo
 - Es. la Coda Ready partizionata in diverse code, e.g.:
 - ▶ **foreground** (interattiva)
 - ▶ **background** (batch)
- Processi assegnati alle code in modo fisso (non passaggio tra code)
- Ogni coda ha un suo algoritmo di scheduling:
 - foreground – RR
 - background – FCFS
- Scheduling tra code:
 - Scheduling a priorità fissata (i.e., serve tutti dal foreground e poi dal background). Possibilità di starvation.
 - Altra possibilità time-slice tra code – ogni coda prende un certo quanto di CPU che può schedulare tra i suoi processi
 - ▶ es., 80% ai processi foreground in RR 20% ai processi in background in FCFS

Code Multiple

highest priority



lowest priority

Code Multiple con Feedback

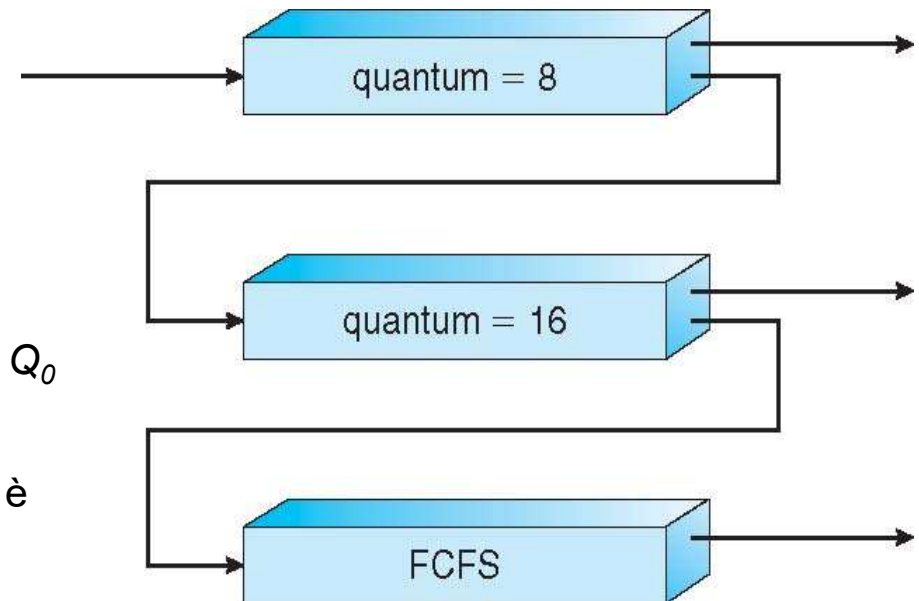
- Metodi più adattivi
- **Un processo può spostarsi tra le code**
 - Processi I/O bound con priorità più elevata
 - Aging (aumento di priorità nel tempo) per evitare starvation
- Lo scheduler di code multiple con feedback definito da:
 - Numero di code
 - Algoritmi di scheduling per ogni coda
 - Metodi per determinare quando aumentare la priorità di processo
 - Metodi per determinare quando abbassare la priorità un processo
 - Metodi per determinare quale coda assegnare ad un processo quando richiede un servizio
- **È lo schedulatore più complesso e flessibile**

Esempio di Coda Multiple con Feedback

- Si considerino tre code:
 - Q_0 – RR quanto di tempo 8 msec
 - Q_1 – RR quanto di tempo 16 msec
 - Q_2 – FCFS

- Scheduling

- Prelazione con priorità 0, 1, 2
- Nuovo processo in Ready messo in coda Q_0
 - ▶ Il processo riceve 8 msec di CPU
 - ▶ Se non finisce in 8 msec, il processo è mosso in coda Q_1
- Se vuota Q_0
 - ▶ il primo processo in Q_1 riceve 16 msec
 - ▶ Se non completa viene prelazonato e si sposta sulla coda Q_2
- Se vuote Q_0 e Q_1 vengono eseguiti i processi in coda Q_2 con FCFS
- Scheduler che dà priorità a processi rapidi e mette in coda quelli più lunghi



Esercizio Scheduling

Si considerino i processi in tabella con tempi di arrivo (in millisecondi) e CPU-brust indicato

Si calcoli il tempo medio di attesa e turnaround RR quanto = 4

<i>processo</i>	<i>tempo di arrivo</i>	<i>CPU-burst (millisec.)</i>
<i>A</i>	0	3
<i>B</i>	2	6
<i>C</i>	4	4
<i>D</i>	6	5
<i>E</i>	8	2

Esercizio Scheduling

Si considerino i processi in tabella con tempi di arrivo (in millisecondi) e CPU-brust indicato

Si calcoli il tempo medio di attesa e turnaround RR quanto = 4

<i>processo</i>	<i>tempo di arrivo</i>	<i>CPU-burst (millisec.)</i>
<i>A</i>	0	3
<i>B</i>	2	6
<i>C</i>	4	4
<i>D</i>	6	5
<i>E</i>	8	2

A	B	C	D	B	E	D
3	7	11	15	17	19	20

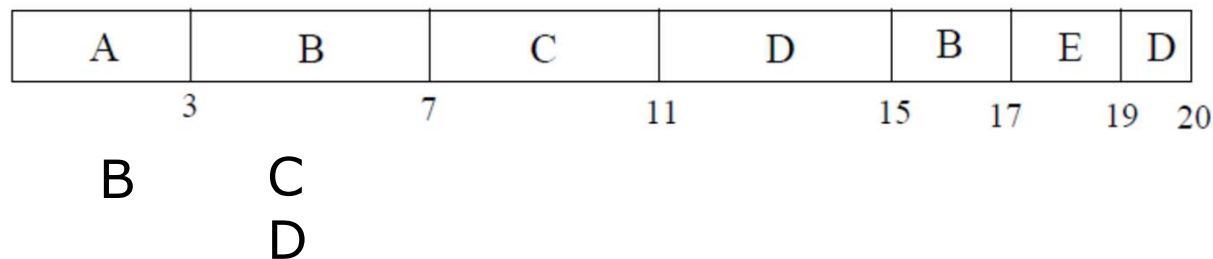
B

Esercizio Scheduling

Si considerino i processi in tabella con tempi di arrivo (in millisecondi) e CPU-burst indicato

Si calcoli il tempo medio di attesa e turnaround RR quanto = 4

<i>processo</i>	<i>tempo di arrivo</i>	<i>CPU-burst (millisec.)</i>
<i>A</i>	0	3
<i>B</i>	2	6
<i>C</i>	4	4
<i>D</i>	6	5
<i>E</i>	8	2

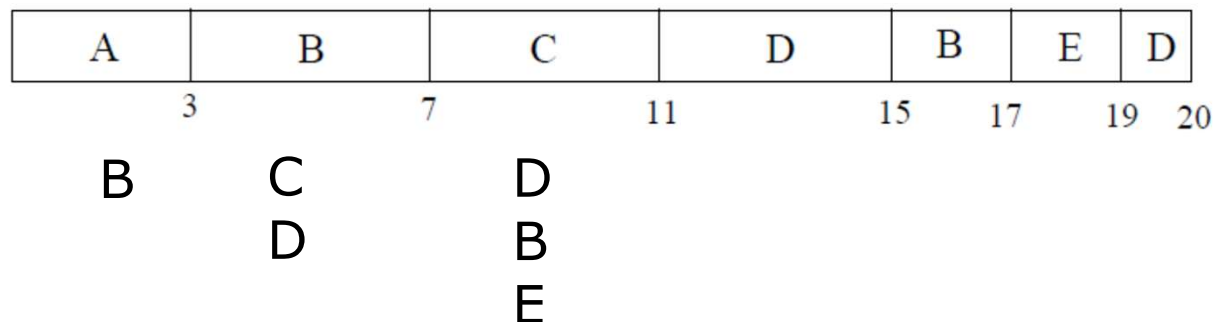


Esercizio Scheduling

Si considerino i processi in tabella con tempi di arrivo (in millisecondi) e CPU-burst indicato

Si calcoli il tempo medio di attesa e turnaround RR quanto = 4

<i>processo</i>	<i>tempo di arrivo</i>	<i>CPU-burst (millisec.)</i>
<i>A</i>	0	3
<i>B</i>	2	6
<i>C</i>	4	4
<i>D</i>	6	5
<i>E</i>	8	2

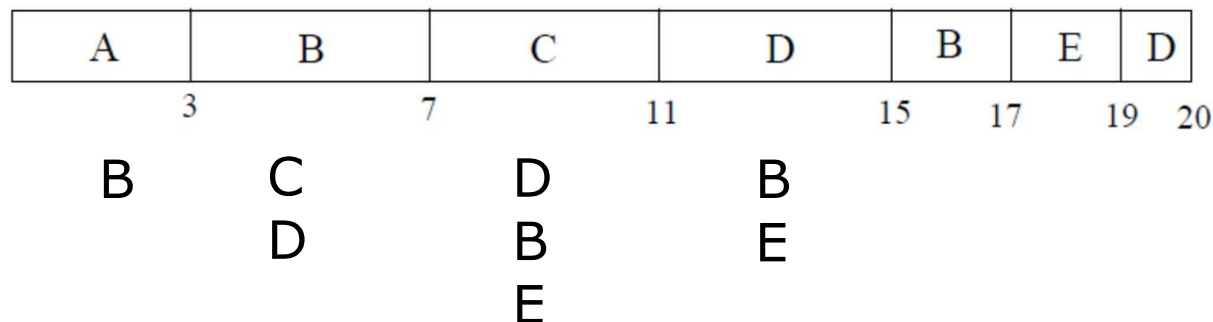


Esercizio Scheduling

Si considerino i processi in tabella con tempi di arrivo (in millisecondi) e CPU-burst indicato

Si calcoli il tempo medio di attesa e turnaround RR quanto = 4

<i>processo</i>	<i>tempo di arrivo</i>	<i>CPU-burst (millisec.)</i>
<i>A</i>	0	3
<i>B</i>	2	6
<i>C</i>	4	4
<i>D</i>	6	5
<i>E</i>	8	2

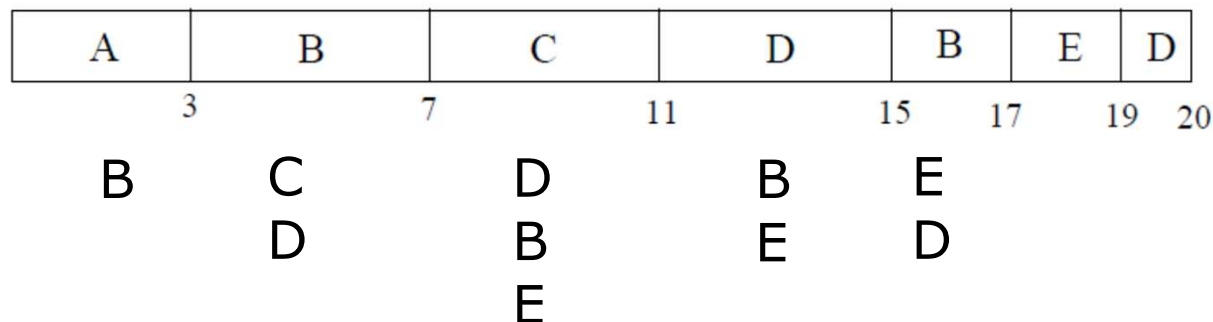


Esercizio Scheduling

Si considerino i processi in tabella con tempi di arrivo (in millisecondi) e CPU-burst indicato

Si calcoli il tempo medio di attesa e turnaround RR quanto = 4

<i>processo</i>	<i>tempo di arrivo</i>	<i>CPU-burst (millisec.)</i>
<i>A</i>	0	3
<i>B</i>	2	6
<i>C</i>	4	4
<i>D</i>	6	5
<i>E</i>	8	2

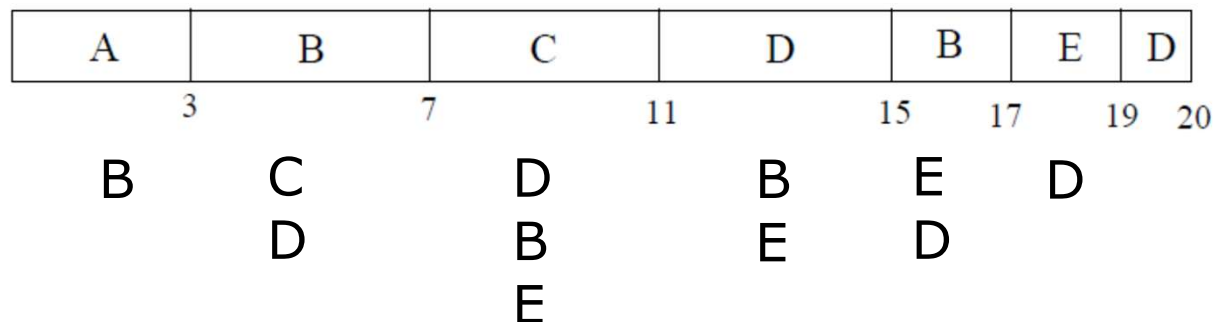


Esercizio Scheduling

Si considerino i processi in tabella con tempi di arrivo (in millisecondi) e CPU-burst indicato

Si calcoli il tempo medio di attesa e turnaround RR quanto = 4

<i>processo</i>	<i>tempo di arrivo</i>	<i>CPU-burst (millisec.)</i>
<i>A</i>	0	3
<i>B</i>	2	6
<i>C</i>	4	4
<i>D</i>	6	5
<i>E</i>	8	2



Esercizio Scheduling

Si considerino i processi in tabella con tempi di arrivo (in millisecondi) e CPU-burst indicato

Si calcoli il tempo medio di attesa e turnaround RR quanto = 4

<i>processo</i>	<i>tempo di arrivo</i>	<i>CPU-burst (millisec.)</i>
<i>A</i>	0	3
<i>B</i>	2	6
<i>C</i>	4	4
<i>D</i>	6	5
<i>E</i>	8	2

A	B	C	D	B	E	D
3	7	11	15	17	19	20

$$t_a(RR-4) = \frac{(3-3-0) + (17-6-2) + (11-4-4) + (20-5-6) + (19-2-8)}{5} = 6$$

Esercizio Scheduling

Si considerino i processi in tabella con tempi di arrivo (in millisecondi) e CPU-burst indicato

Si calcoli il tempo medio di attesa e turnaround RR quanto = 4

<i>processo</i>	<i>tempo di arrivo</i>	<i>CPU-burst (millisec.)</i>
<i>A</i>	0	3
<i>B</i>	2	6
<i>C</i>	4	4
<i>D</i>	6	5
<i>E</i>	8	2

A	B	C	D	B	E	D
3	7	11	15	17	19	20

$$t_{tr}(RR - 4) = \frac{(3 - 0) + (17 - 2) + (11 - 4) + (20 - 6) + (19 - 8)}{5} = 10$$

Esercizio Scheduling

Si considerino i processi in tabella con tempi di arrivo (in millisecondi) e tempo di esecuzione

<i>processo</i>	<i>tempo di arrivo</i>	<i>tempo di esecuzione</i>
P_1	0	20
P_2	8	5
P_3	3	12
P_4	10	6
P_5	7	8

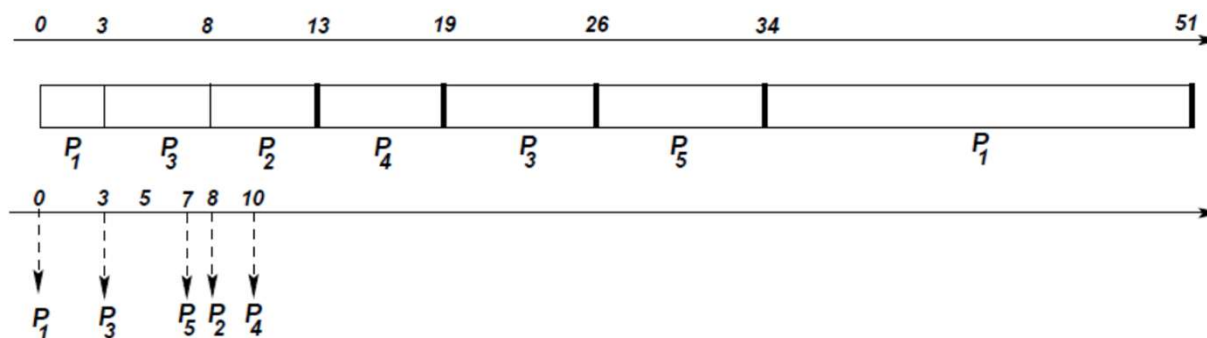
Calcolare tempo medio di attesa e turnaround per SJF preemptive

Esercizio Scheduling

Si considerino i processi in tabella con tempi di arrivo (in millisecondi) e tempo di esecuzione

<i>processo</i>	<i>tempo di arrivo</i>	<i>tempo di esecuzione</i>
P_1	0	20
P_2	8	5
P_3	3	12
P_4	10	6
P_5	7	8

Calcolare tempo medio di attesa e turnaround per SJF preemptive

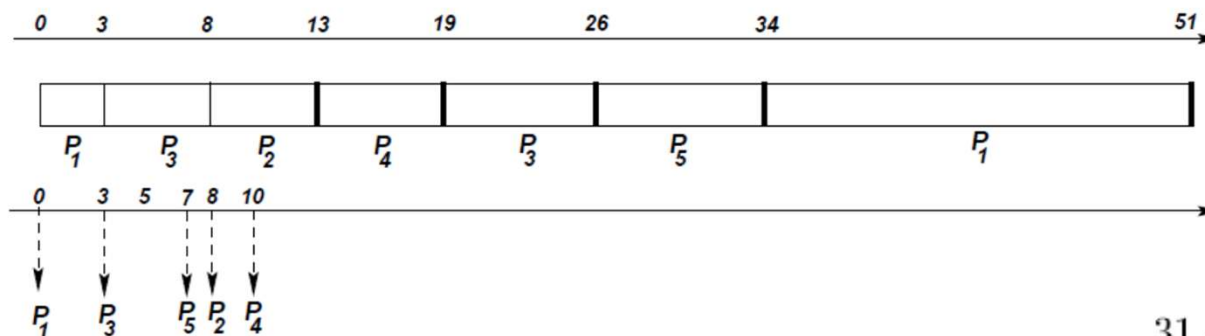


Esercizio Scheduling

Si considerino i processi in tabella con tempi di arrivo (in millisecondi) e tempo di esecuzione

<i>processo</i>	<i>tempo di arrivo</i>	<i>tempo di esecuzione</i>
P_1	0	20
P_2	8	5
P_3	3	12
P_4	10	6
P_5	7	8

Calcolare tempo medio di attesa e turnaround per SJF preemptive



$$t_a = \frac{31 + 0 + 11 + 3 + 19}{5} = 12.8 \text{ msec}$$

$$t_{tr} = \frac{51 + 5 + 23 + 9 + 27}{5} = 23 \text{ msec}$$

Scheduling di Thread

- ❑ Distinguere tra thread **user-level** e **kernel-level**
- ❑ Moderni SO schedulano **thread kernel-level**, non processi
- ❑ Thread user-level gestiti da libreria e kernel non ne è al corrente
- ❑ Per essere eseguiti i thread user-level devono essere mappati in kernel-level

Scheduling di Thread

- Per essere eseguiti i thread user-level devono essere mappati in kernel-level
- Modelli many-to-one e many-to-many, la libreria dei thread gestisce gli user-level threads per poi lanciare i lightweight processes (LWPs)
 - Lo schema è detto **Process-Contention Scope (PCS)**: la competizione per lo CPU è tra thread dello stesso processo
 - Tipicamente scheduling con priorità (definita dal programmatore)
- Per decidere il kernel thread da schedulare in CPU il kernel usa un **System-Contention Scope (SCS)**
 - Competizioni tra tutti i thread nel sistema
 - Sistemi che utilizzano modelli one-to-one (Linux, Windows) usano SCS

Pthread Scheduling

- API permettono di specificare PCS o SCS durante la creazione dei thread
- POSIX Pthread permette di specificare entrambe le modalità:
 - PTHREAD_SCOPE_PROCESS usa PCS scheduling
 - PTHREAD_SCOPE_SYSTEM usa SCS scheduling
- Limitato da OS – Linux e Mac OS permettono solo PTHREAD_SCOPE_SYSTEM

Pthread Scheduling API

```
#include <pthread.h>
#include <stdio.h>
#define NUM_THREADS 5
int main(int argc, char *argv[]) {
    int i, scope;
    pthread_t tid[NUM_THREADS];
    pthread_attr_t attr;
    /* get the default attributes */
    pthread_attr_init(&attr);
    /* first inquire on the current scope */
    if (pthread_attr_getscope(&attr, &scope) != 0)
        fprintf(stderr, "Unable to get scheduling scope\n");
    else {
        if (scope == PTHREAD_SCOPE_PROCESS)
            printf("PTHREAD_SCOPE_PROCESS");
        else if (scope == PTHREAD_SCOPE_SYSTEM)
            printf("PTHREAD_SCOPE_SYSTEM");
        else
            fprintf(stderr, "Illegal scope value.\n");
    }
}
```

Pthread Scheduling API

```
/* set the scheduling algorithm to PCS or SCS */
pthread_attr_setscope(&attr, PTHREAD_SCOPE_SYSTEM);
/* create the threads */
for (i = 0; i < NUM_THREADS; i++)
    pthread_create(&tid[i], &attr, runner, NULL);
/* now join on each thread */
for (i = 0; i < NUM_THREADS; i++)
    pthread_join(tid[i], NULL);
}

/* Each thread will begin control in this function */
void *runner(void *param)
{
    /* do some work ... */
    pthread_exit(0);
}
```

Scheduling Multi-Processore

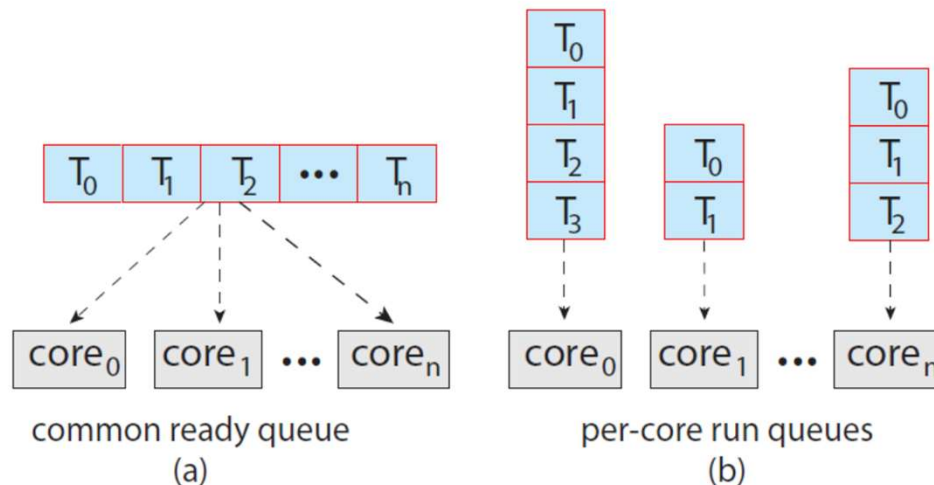
- CPU scheduling più complesso se ci sono più CPU
- Diverse architetture disponibili:
 - **Multicore CPU**
 - **Multithreaded cores**
 - **Sistemi NUMA**
 - **Multiprocessamento eterogeneo**
- Per i primi tre casi assumiamo processori omogenei (stesse capacità)

Scheduling Multi-Processore

- CPU scheduling più complesso se ci sono più processori
- **Homogeneous processors** – processori identici per funzionalità
- **Asymmetric multiprocessing** – un solo processore (master) coinvolto nelle decisioni di scheduling e accede alle strutture dati di sistema senza condivisione di dati
 - Semplifica la gestione, ma il processore server master fa da collo di bottiglia
- **Symmetric multiprocessing (SMP)** – approccio standard: ogni processore fa self-scheduling

Scheduling Multi-Processore

- CPU scheduling più complesso se ci sono più CPU
- **Homogeneous processors** – processori identici per funzionalità
- **Asymmetric multiprocessing** – un solo processore (master) prende decisioni di scheduling e accede alle strutture dati di sistema senza condivisione di dati.
 - Semplifica la gestione, ma il processore server master fa da collo di bottiglia
- **Symmetric multiprocessing (SMP)** – approccio standard: ogni processore fa self-scheduling, due modalità
 - tutti i processi in una sola coda ready
 - ognuno ha una sua coda privata di processi ready



Scheduling Multi-Processore

- ❑ CPU scheduling più complesso se ci sono più CPU
- ❑ **Homogeneous processors** – processori identici per funzionalità
- ❑ **Asymmetric multiprocessing** – un solo processore (master) prende decisioni di scheduling e accede alle strutture dati di sistema senza condivisione di dati. Semplifica la gestione, ma il processore server master fa da collo di bottiglia
- ❑ **Symmetric multiprocessing (SMP)** – approccio standard: ogni processore fa self-scheduling
 - ❑ tutti i processi in una coda ready oppure ognuno ha una sua coda privata di processi ready
 - ❑ Attualmente la soluzione più comune/standard in SMP sono le **code private**
 - ❑ Tutti i sistemi supportano SMP (Linux, Windows, MacOS, Android, etc.)

Scheduling Multi-Processore

- ❑ CPU scheduling più complesso se ci sono più CPU
- ❑ **Homogeneous processors** – processori identici per funzionalità
- ❑ **Asymmetric multiprocessing** – un solo processore (master) prende decisioni di scheduling e accede alle strutture dati di sistema senza condivisione di dati. Semplifica la gestione, ma il processore server master fa da collo di bottiglia
- ❑ **Symmetric multiprocessing (SMP)** – approccio standard: ogni processore fa self-scheduling
 - ❑ tutti i processi in una coda ready oppure ognuno ha una sua coda privata di processi ready
 - ❑ Attualmente la soluzione più comune/standard in SMP sono le code private
 - ❑ Tutti i sistemi supportano SMP (Linux, Windows, MacOS, Android, etc.)